David Mohaisen, Professor
University of Central Florida
Department of Computer Science
Cybersecurity and Privacy Cluster

https://cs.ucf.edu/~mohaisen/

Sequence-to-Sequence Models

① Bidirectional RNNs
② Stacked RNNs
③ Encoder–Decoder Models
④ Attention Mechanisms
⑤ Neural Machine Translation
⑥ Seq2Seq Challenges

How to Evaluate LMs

- **Perplexity (PPL)**
  - Measures how well a model predicts a sequence.
  - Lower **perplexity** → better model performance.
  - Common for **probabilistic language models** (e.g., RNNs, LSTMs, Transformers).

$$PPL = 2^{(-\frac{1}{N}\sum \log_2 P(w_i))}$$

- **BLEU (Bilingual Evaluation Understudy)**
  - Compares generated text to reference texts using **n-gram precision**.
  - Higher **BLEU** → better match with human-written text.
  - Used in **machine translation, summarization, and text generation**.

How to Evaluate LMs

- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)**
  - Measures **recall-based overlap** between generated text and reference text.
  - **ROUGE-N**: n-gram recall (e.g., ROUGE-1, ROUGE-2).
  - **ROUGE-L**: Longest common subsequence (LCS) matching.
  - Used for **summarization & text coherence evaluation**.
- **METEOR (Metric for Evaluation of Translation with Explicit ORdering)**
  - Improves BLEU by considering **synonyms & stemming**.
  - Higher correlation with human judgment than BLEU.
  - Used for **machine translation and paraphrase evaluation**.
- **BERTScore**
  - Uses **pretrained embeddings** (e.g., BERT) to compare similarity.
  - Considers **semantic meaning** rather than just word overlap.
  - Used for **advanced NLP evaluation (e.g., dialogue, summarization, translation)**.

How to Evaluate LMs

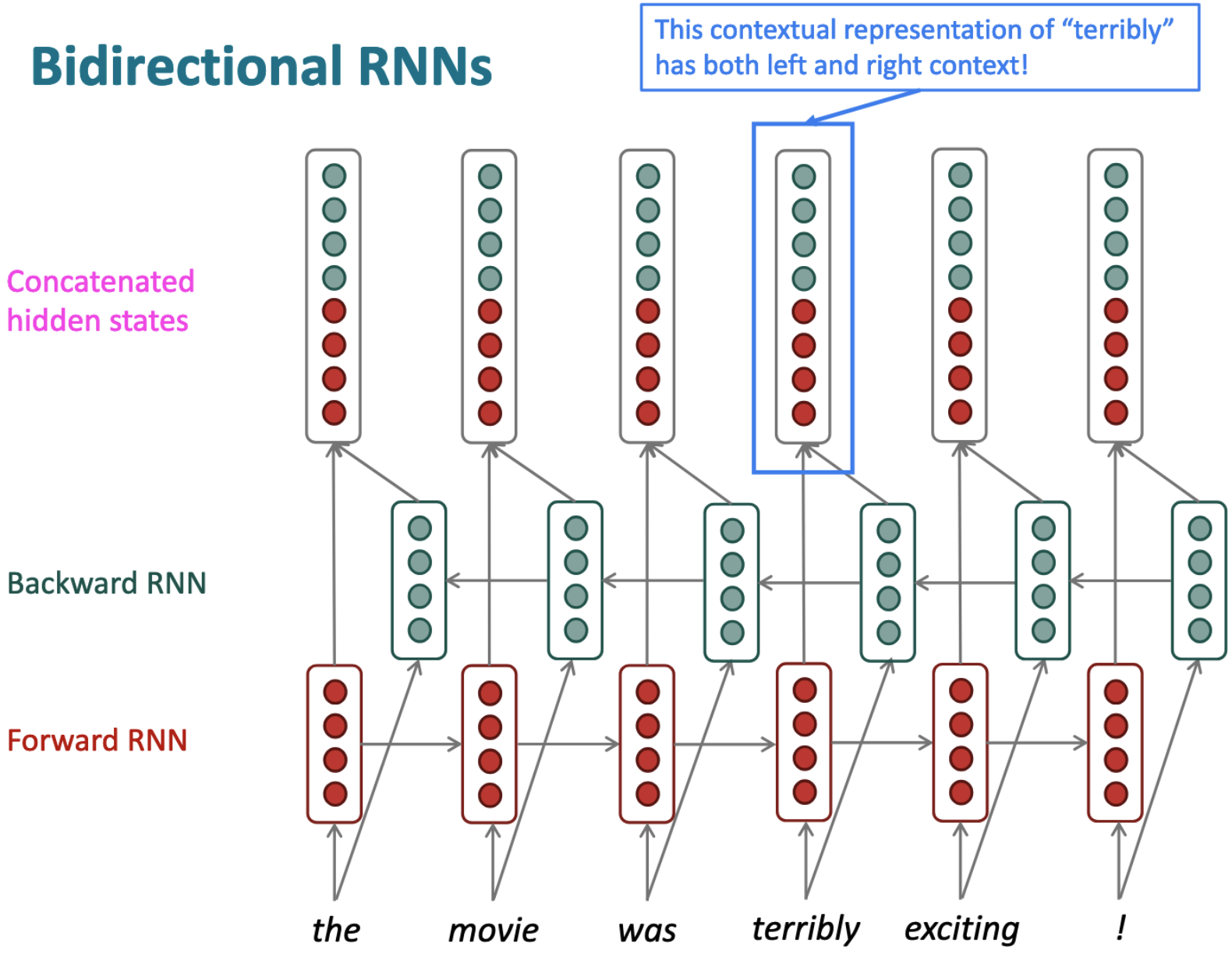
- **Human Evaluation**
  - **Fluency**: How natural and grammatically correct is the output?
  - **Coherence**: Does the text make sense contextually?
  - **Relevance**: Is the output meaningful to the input prompt?
- **Real-World Testing**
  - **A/B Testing**: Compare model versions in real applications.
  - **User Engagement**: Track user preferences & satisfaction.
  - **Task-Specific Accuracy**: Measure success in applications like chatbots or summarization.

Bidirectional RNNs

- **What are Bidirectional RNNs?**
  - BRNNs are a type of **Recurrent Neural Network (RNN)** designed to process sequences in **both forward and backward directions**.
  - Unlike standard RNNs, which process input sequentially in one direction, BRNNs **capture both past and future context** at each time step.
  - Introduced by **Schuster & Paliwal (1997)** for sequence-based tasks.
- **Why BRNNs?**
  - Many NLP tasks require **understanding the full context of a word or sentence** (e.g., Named Entity Recognition, Speech Recognition).
  - A traditional **unidirectional RNN** can only use **past information** when predicting, limiting its effectiveness.
  - BRNNs **enhance representation learning** by integrating **both past and future** dependencies in a sequence.

Bidirectional RNNs, Operation

- **Architecture and Operation**
  - A BRNN consists of **two RNN layers**:
    - **Forward RNN**: Processes the sequence from **left to right**.
    - **Backward RNN**: Processes the sequence from **right to left**.
  - The outputs of both RNNs are **concatenated** at each time step: ht=concat(h_t^{forward}, h_t^{backward})
  - This allows the model to leverage **past and future context simultaneously**.



On timestep *t*:

This is a general notation to mean "compute one forward step of the RNN" – it could be a simple RNN or LSTM computation.

Forward RNN $\overrightarrow{\boldsymbol{h}}^{(t)} = \boxed{\mathrm{RNN}_{\mathrm{FW}}}(\overrightarrow{\boldsymbol{h}}^{(t-1)}, \boldsymbol{x}^{(t)})$

Backward RNN $\overleftarrow{\boldsymbol{h}}^{(t)} = \mathrm{RNN}_{\mathrm{BW}}(\overleftarrow{\boldsymbol{h}}^{(t+1)}, \boldsymbol{x}^{(t)})$

Generally, these two RNNs have separate weights

Concatenated hidden states $\boxed{\boldsymbol{h}^{(t)}} = [\overrightarrow{\boldsymbol{h}}^{(t)}; \overleftarrow{\boldsymbol{h}}^{(t)}]$

We regard this as "the hidden state" of a bidirectional RNN. This is what we pass on to the next parts of the network.

Bidirectional RNN, Use Cases

- **Use Cases in NLP**
  - **Speech Recognition**: Understanding a word's pronunciation requires considering both its **preceding and following sounds**.
  - **Named Entity Recognition (NER)**: Identifying names (e.g., "Apple" as a company) depends on **both preceding and following words**.
  - **Part-of-Speech Tagging**: A word's POS tag often depends on **the words before and after it** in a sentence.
  - **Machine Translation**: Context from **entire sentences** improves translation quality.

Bidirectional RNNs, Challenges

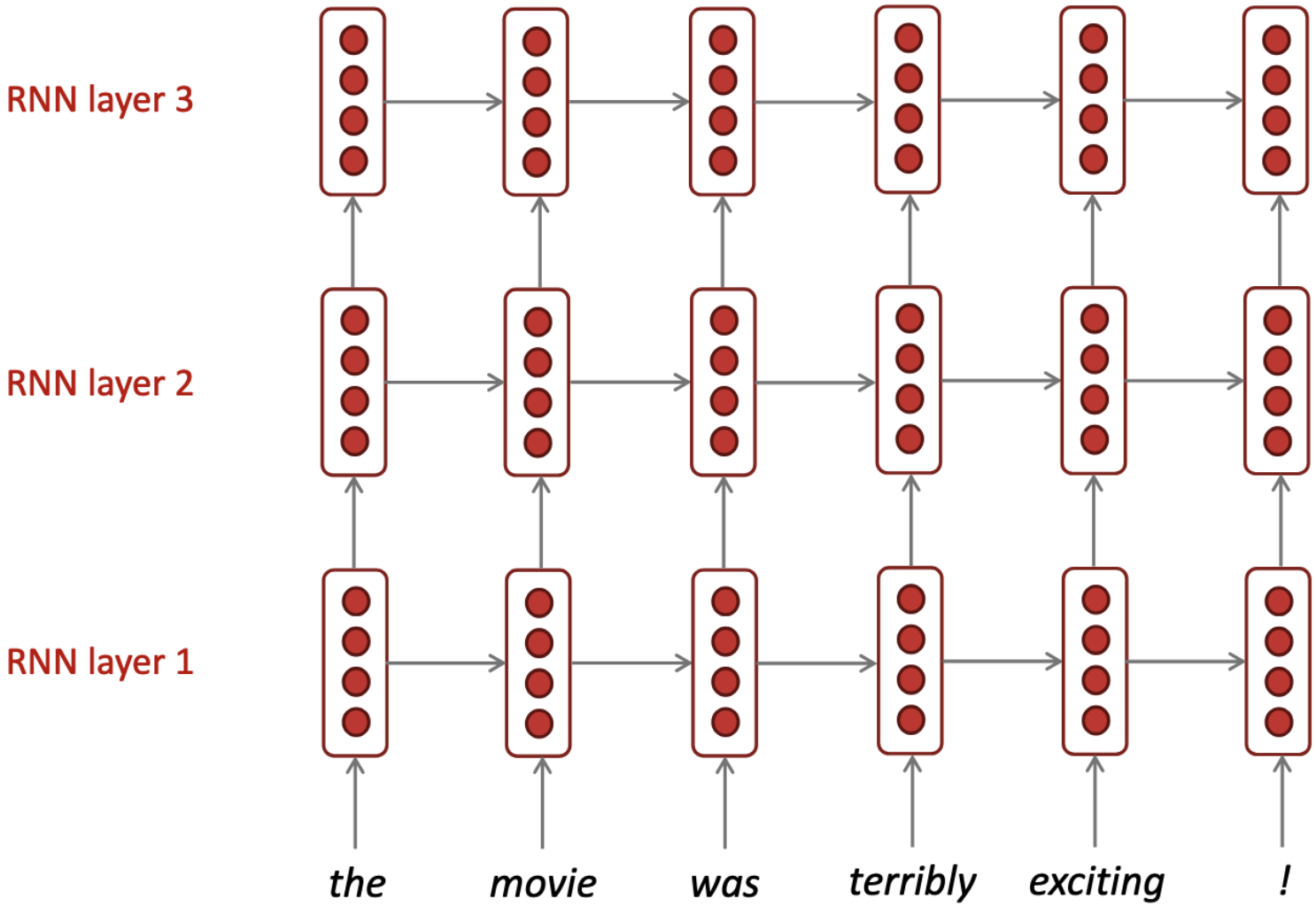
- **High Computational Cost**:
  - BRNNs **double the number of parameters** compared to unidirectional RNNs.
  - This increases memory and computation requirements.
  - **Solution**: Use optimized architectures like **Bidirectional LSTMs or GRUs** for efficiency.
- **Not Suitable for Real-Time Processing**:
  - Since BRNNs require **access to the full sequence**, they **cannot be used for online/real-time tasks** (e.g., streaming speech recognition).
  - **Solution**: Hybrid approaches like **Transformer-based models** (e.g., BERT) offer alternatives with better efficiency.
- **Vanishing Gradient Problem**:
  - Like traditional RNNs, BRNNs suffer from **difficulty in learning long-range dependencies**.
  - **Solution**: Using **LSTMs or GRUs** mitigates this issue by introducing **gated mechanisms** to control information flow.

Multi-layer RNNs

- Multi-layer RNNs (also called **stacked RNNs**) consist of **multiple recurrent layers** stacked on top of each other.
- Each layer processes the sequence and **passes its hidden states as input next layer**.
- Multi-layer RNNs **learn more abstract representations** of the data.
- **Why Use Multi-Layer RNNs?**
  - Single-layer RNNs struggle to capture **complex hierarchical structures**.
  - Multi-layer architectures help in:
    - Learning **low-level** patterns in **lower layers** (e.g., phonemes in speech).
    - Learning **high-level** representations in **upper layers** (e.g., meaning and semantics).
  - Used extensively in **speech recognition, machine translation, and text generation**.

How Multi-Layer RNNs Work

- The network consists of **L stacked layers**, where each layer receives input from the previous layer: $h_t^{(l)}=f(W^{(l)}h_t^{(l-1)}+b^{(l)})$ where:
  - $h_t^{(l)}$ is the hidden state at layer l.
  - $W^{(l)}$ are learnable weights.
  - f is a non-linearity (e.g., tanh or ReLU).



How Multi-Layer RNNs Work

- **Deep Feature Extraction**
  - **First layer** learns **local dependencies** (e.g., word-level patterns).
  - **Middle layers** refine representations, detecting **phrases and syntax**.
  - **Top layer** captures **global dependencies**, improving **semantic understanding**.
- **Use Cases in NLP**
  - **Speech Recognition**: Captures both short-term phonetic info and long-term speech patterns.
  - **Text Classification**: Hierarchically learns word and sentence representations.
  - **Machine Translation**: Deep architectures improve alignment and contextual understanding.

Challenges in Multi-Layer RNNs

- **Vanishing Gradient Problem**
  - Gradients diminish as they backpropagate through multiple layers.
  - **Solution**: Use **LSTMs/GRUs**, which handle long-range dependencies better.
- **Increased Computational Cost**
  - More layers mean higher memory and processing requirements.
  - **Solution**: Use **truncated backpropagation** and optimize architectures (e.g., **attention mechanisms**).
- **Optimization Difficulty**
  - Stacking too many layers can cause overfitting or difficulty in convergence.
  - **Solution**: Apply **dropout** and **batch normalization** to regularize training.
- **Not Ideal for Very Long Sequences**
  - Even with deep architectures, RNNs struggle with extremely long dependencies.
  - **Solution**: Use **Transformers or attention-based models (e.g., BERT, GPT)** for long-range dependency modeling.

Multi-layer RNNs in Practice

- Multi-layer or stacked RNNs allow a network to compute more complex representations
  - they work better than just have one layer of high-dimensional encodings!
  - The lower RNNs should compute lower-level features and the higher RNNs should compute higher-level features.
- High-performing RNNs are usually multi-layer (but aren't as deep as convolutional or feed-forward networks)
- For Neural Machine Translation, 2 to 4 layers is best for the encoder RNN, and 4 layers is best for the decoder RNN
  - Often 2 layers is a lot better than 1, and 3 might be a little better than 2
  - Usually, skip-connections/dense-connections are needed to train deeper RNNs (e.g., 8 layers)
- Transformer-based networks (e.g., BERT) are usually deeper, like 12 or 24 layers.

Encoder-Decoder Architectures

- The **Encoder-Decoder architecture** is a foundational deep learning framework used for **sequence-to-sequence (Seq2Seq) tasks**, such as **machine translation, text summarization, and speech recognition**. It consists of two key components: Encoder and Decoder.

The Encoder

- The **encoder** processes the input sequence (e.g., a sentence in English) and **compresses** it into a **fixed-length vector representation** (also called a **context vector** or **hidden state**).
- Typically, the encoder is a **Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM), or Gated Recurrent Unit (GRU)**.
- **Steps in the Encoder:**
  - Each **word** in the input sequence is converted into an **embedding vector**.
  - The RNN processes the sequence **word by word**, updating its hidden state at each step: $h_t=f(W_x x_t+W_h h_{t-1})$ where h_t is the hidden state at time ttt.
  - After processing the last word, the encoder outputs a **final hidden state** h_T, which serves as the compressed representation of the entire input sequence.
- **Key Limitation:**
  - Traditional Seq2Seq models **compress all information into a single vector**.
  - This can be problematic for **long sequences**, leading to **loss of information**.

The Decoder

- The **decoder** takes the **context vector** (the final hidden state from the encoder) and generates the **translated output sequence** (e.g., a sentence in French).
- Like the encoder, the decoder is also an **RNN, LSTM, or GRU** but operates **autoregressively**, meaning:
  - It **receives** the encoded context vector and an initial hidden state.
  - It **predicts** the first word of the output sequence.
  - This **predicted word** is then **fed back** as input for the next time step.
  - The process repeats **until a stop condition is met** (e.g., an <EOS> token).
- Mathematically: $s_t = g(W_s y_{t-1} + U s_{t-1} + V h_T)$:
  - s_t is the hidden state of the decoder at step t,
  - y_{t-1} is the previously generated word,
  - h_T is the encoder's final hidden state.
- **Limitation:** If the entire sentence is compressed into **one vector**, the model struggles with **long sentences**.

Attentions and Applications

- **Attention Mechanism: Enhancing the Encoder-Decoder Model**
  - To **overcome the information bottleneck**, **attention mechanisms** were introduced.
  - Instead of using only **one fixed context vector**, **attention allows the decoder to focus on different parts of the input sequence** at each step.
  - The decoder **assigns different attention weights** to different words in the input, dynamically adjusting its focus.
- Use Cases of Encoder-Decoder Architecture
  - **Machine Translation** (Google Translate, DeepL)
  - **Text Summarization** (news summarization)
  - **Speech-to-Text** (ASR systems)
  - **Image Captioning** (describing images in text)

Machine Translation

- **Machine Translation (MT)** is the process of automatically translating text from one language to another using computational models.
  - It enables **cross-lingual communication** and is widely used in applications like **Google Translate, DeepL, and Microsoft Translator**.
- **Types of Machine Translation**
  1. **Rule-Based MT (RBMT)**: Uses hand-crafted linguistic rules.
  2. **Statistical MT (SMT)**: Uses probabilities from bilingual corpora.
  3. **Neural MT (NMT)**: Leverages deep learning to generate translations.
- **Why is MT Challenging?**
  - **Syntax and Grammar Differences**: Sentence structures vary across languages.
  - **Word Ambiguity**: A single word may have multiple meanings.
  - **Context Dependence**: Meaning depends on broader sentence context.
  - **Idiomatic Expressions**: Direct translations often fail.

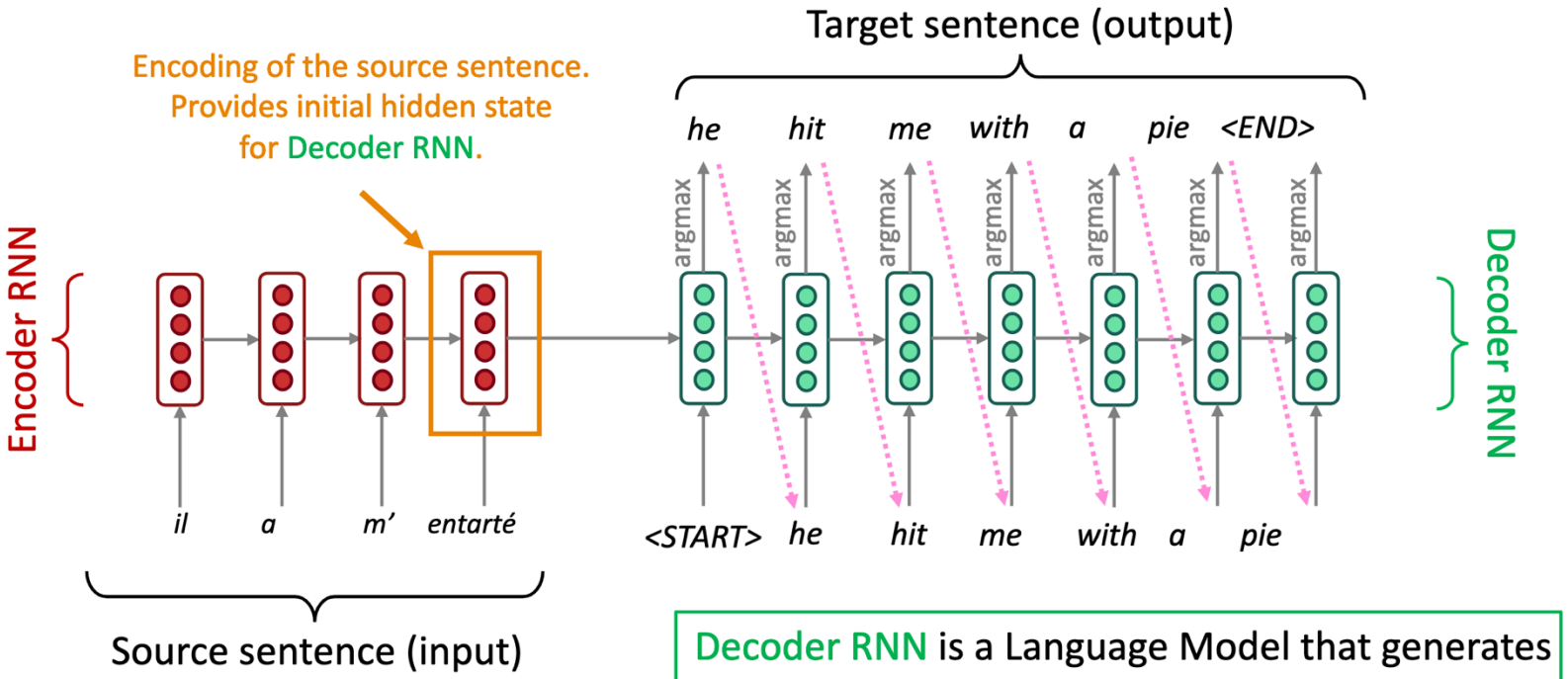
Neural Machine Translation (NMT) and Sequence-to-Sequence Models

- **How Neural MT Works**
  - **NMT replaces traditional phrase-based translation with deep learning models.**
  - Uses an **Encoder-Decoder** architecture, often with attention mechanisms.
- **Sequence-to-Sequence (Seq2Seq) Model**
  - **Encoder**: Converts input text (e.g., English sentence) into a fixed-length vector.
  - **Decoder**: Generates the translated output (e.g., French sentence) from this vector.
  - **Attention Mechanism**: Allows the model to focus on relevant words during translation.
- **Advantages of NMT**
  - **Captures long-range dependencies** in text.
  - **Handles contextual meaning better** than SMT and RBMT.
  - **Produces fluent and human-like translations**.

Neural Machine Translation (NMT) and Sequence-to-Sequence Models

- The sequence-to-sequence model is an example of a Conditional Language Model
  - Language Model because the decoder is predicting the next word of the target sentence y
  - Conditional because its predictions are also conditioned on the source sentence x
- NMT directly calculates $P(y|x)$
  - $P(y|x) = P(y_1|x)P(y_2|y_1,x)\ldots P(y_T|y_1,..,y_{T-1},x)$
- How to train NMT systems? Get a big parallel corpus

Neural Machine Translation (NMT) and Sequence-to-Sequence Models

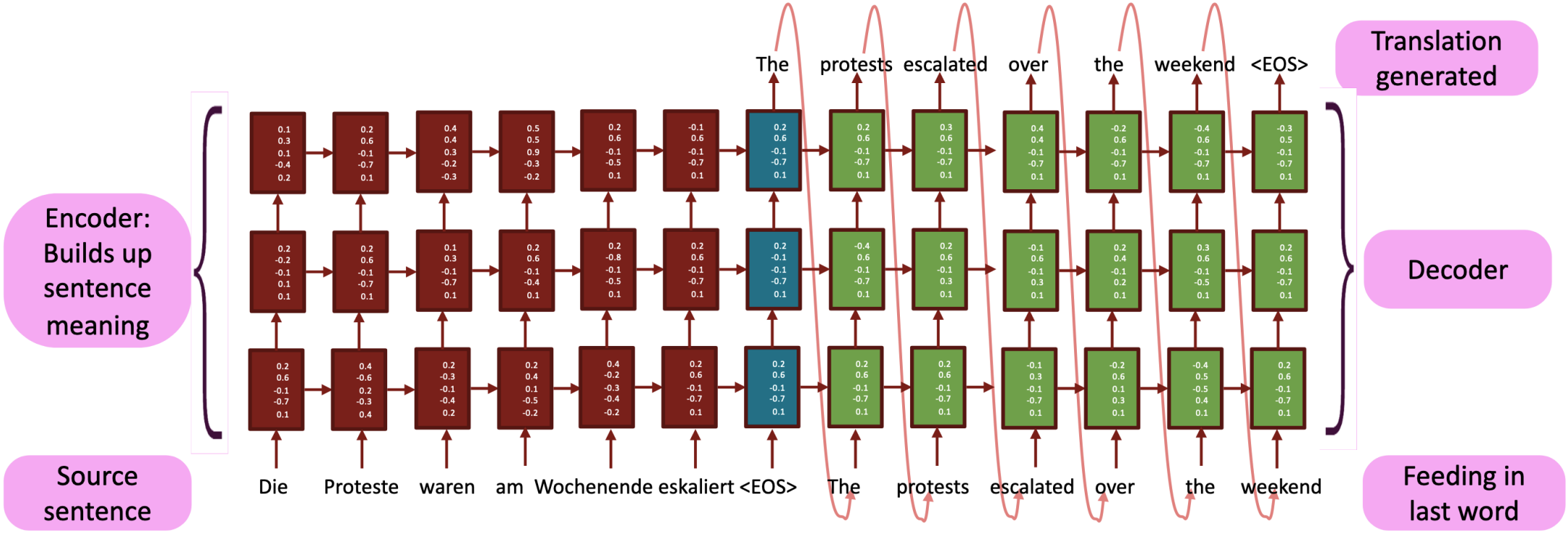


Decoder RNN is a Language Model that generates target sentence, *conditioned on encoding*.

Encoder RNN produces an encoding of the source sentence.

Note: This diagram shows **test time** behavior: decoder output is fed in ·····→ as next step's input

Neural Machine Translation (NMT) and Sequence-to-Sequence Models



Challenges in NMT

- **Current Challenges in NMT**
  - **Rare and Low-Resource Languages**: Limited training data for many languages.
  - **Handling Long Sentences**: Longer inputs can cause errors in coherence.
  - **Cultural and Contextual Understanding**: Models struggle with idioms, metaphors, and cultural nuances.
  - **Computational Cost**: Training large NMT models requires substantial resources.
  - Hard to interpret/debug, difficult to control (guidelines), safety concerns.
- **Future Trends in MT**
  - **Multilingual Models**: Unified models that translate multiple languages (e.g., Google's mT5, Meta's NLLB).
  - **Zero-Shot Translation**: Translation without parallel training data for a language pair.
  - **Better Context Awareness**: Incorporating world knowledge and discourse-level information.
  - **Hybrid Approaches**: Combining **rule-based, statistical, and neural methods** for improved accuracy.